

Case Study 1: Online Shopping Order System

Business Scenario

An e-commerce company stores customer orders in MongoDB.
Each customer document contains embedded product order details.

The admin wants reports like:

- ✓ total amount spent per customer
- ✓ average product price
- ✓ highest and lowest product price
- ✓ number of customers per product

Table View (Logical)

orders

customer_id	name	items (array)
C101	Asha	[{ product: Mobile, price: 15000 }, { product: Earphones, price: 2000 }]
C102	Kiran	[{ product: Mobile, price: 15000 }, { product: Laptop, price: 50000 }]
C103	Neha	[{ product: Earphones, price: 2000 }, { product: Laptop, price: 50000 }]

Sample Data

```
db.orders.insertMany([
  {
    customer_id: "C101",
    name: "Asha",
    items: [
      { product: "Mobile", price: 15000 },
      { product: "Earphones", price: 2000 }
    ]
  },
  {
    customer_id: "C102",
    name: "Kiran",
    items: [
      { product: "Mobile", price: 15000 },
      { product: "Laptop", price: 50000 }
    ]
  }
])
```

```
    ]
  },
  {
    customer_id: "C103",
    name: "Neha",
    items: [
      { product: "Earphones", price: 2000 },
      { product: "Laptop", price: 50000 }
    ]
  }
]);
```

SCENARIO 1: Mobile Product Revenue Report

Scenario

The company wants a report of **Mobile product orders** with these conditions:

Filters:

- ✓ Product must be Mobile
- ✓ Price must be ≥ 15000

They want:

- ✓ Total customers who bought Mobile
- ✓ Total revenue
- ✓ Average, minimum, maximum price
- ✓ List of customer names
- ✓ First customer who ordered

Store result in collection:

mobile_report

SCENARIO 2: High Value Products Report

Scenario

The management wants a report of products where:

Filters:

- ✓ Price ≥ 15000
- ✓ Product = Mobile or Laptop

They want:

- ✓ Number of orders per product
- ✓ Total revenue per product
- ✓ Average, min, max price
- ✓ Customer list per product

Save output into:

high_value_products_report

SCENARIO 3: Customer-wise Spending Report

Scenario

The company wants a report of customers who:

Filters:

- ✓ Have purchased more than one product
- ✓ Total amount spent $\geq$ 30000

They want:

- ✓ Customer name
- ✓ Total amount spent
- ✓ Average price
- ✓ Minimum and maximum product price
- ✓ List of products purchased

Save output into:

customer_spending_report

Case Study 2 : Movie Ticket Booking System

Collections:

customers

movies

bookings

Relationships:

One-to-One $\rightarrow$ Customer $\leftrightarrow$ Profile

One-to-Many $\rightarrow$ Customer $\rightarrow$ Bookings

Many-to-Many $\rightarrow$ Customers $\leftrightarrow$ Movies (via bookings)

Table 1: CUSTOMERS

(One-to-One $\rightarrow$ customer $\leftrightarrow$ profile)

<u>_id</u>	name	profile (embedded)
1	Arun	{ age: 25, city: "Chennai" }
2	Priya	{ age: 22, city: "Bangalore" }
3	Rahul	{ age: 30, city: "Hyderabad" }
4	Neha	{ age: 28, city: "Mumbai" }

5	Kiran	{ age: 35, city: "Pune" }
---	-------	---------------------------

Table 2: MOVIES

_id	title	genre	price
M1	Inception	Sci-Fi	300
M2	Avatar	Fantasy	350
M3	Interstellar	Sci-Fi	400

Table 3: BOOKINGS

_id	customer_id	movie_id	show_date
301	1	M1	2026-02-01
302	1	M2	2026-02-03
303	2	M1	2026-02-05
304	3	M3	2026-02-06
305	3	M1	2026-02-08
306	4	M2	2026-02-10
307	5	M3	2026-02-12
308	5	M2	2026-02-14

✓ Relationships

- ✓ One-to-One → customer → profile
- ✓ One-to-Many → customer → bookings
- ✓ Many-to-Many → customers ↔ movies (via bookings)

Collection 1: customers

```
db.customers.insertMany([
{
  _id: 1,
  name: "Arun",
  profile: { age: 25, city: "Chennai" } // One-to-One
},
{
  _id: 2,
  name: "Priya",
  profile: { age: 22, city: "Bangalore" }
},
{
  _id: 3,
```

```

    name: "Rahul",
    profile: { age: 30, city: "Hyderabad" }
  },
  {
    _id: 4,
    name: "Neha",
    profile: { age: 28, city: "Mumbai" }
  },
  {
    _id: 5,
    name: "Kiran",
    profile: { age: 35, city: "Pune" }
  }
])

```

Collection 2: movies

```

db.movies.insertMany([
  { _id: "M1", title: "Inception", genre: "Sci-Fi", price: 300 },
  { _id: "M2", title: "Avatar", genre: "Fantasy", price: 350 },
  { _id: "M3", title: "Interstellar", genre: "Sci-Fi", price: 400 }
])

```

Collection 3: bookings

```

db.bookings.insertMany([
  { _id: 301, customer_id: 1, movie_id: "M1", show_date: "2026-02-01" },
  { _id: 302, customer_id: 1, movie_id: "M2", show_date: "2026-02-03" },

  { _id: 303, customer_id: 2, movie_id: "M1", show_date: "2026-02-05" },

  { _id: 304, customer_id: 3, movie_id: "M3", show_date: "2026-02-06" },
  { _id: 305, customer_id: 3, movie_id: "M1", show_date: "2026-02-08" },

  { _id: 306, customer_id: 4, movie_id: "M2", show_date: "2026-02-10" },

  { _id: 307, customer_id: 5, movie_id: "M3", show_date: "2026-02-12" },
  { _id: 308, customer_id: 5, movie_id: "M2", show_date: "2026-02-14" }
])

```

Query 1: Total bookings per movie

Goal:

Find how many times each movie was booked.

Result:

For every movie, show the total number of bookings.

Query 2: Movie-wise revenue report

Goal:

Find revenue and price statistics for each movie.

Result:

For each movie, display:

- total tickets sold
- total revenue
- average ticket price
- minimum ticket price
- maximum ticket price

Query 3: Customer-wise spending report

Goal:

Find how much each customer has spent and which movies they watched.

Result:

For each customer, display:

- number of movies booked
- total amount spent
- average spending
- list of movies watched

Query 4: Customers with multiple bookings

Goal:

Identify customers who booked more than one movie.

Result:

List of customers who made multiple bookings.

Query 5: Genre-wise booking count

Goal:

Find which movie genre is most popular.

Result:

For each genre, show the total number of bookings.

Query 6: Movie popularity with customer list

Goal:

Show movie popularity along with customer details.

Result:

For each movie, display:

- total number of bookings
- list of customers who booked that movie